Consistent Hashing

Implementation from First Principles — System Design Fundamentals

1. The Problem with Modulo Hashing

In distributed systems, data needs to be partitioned across multiple servers to handle load and storage effectively. A common and simple approach to determine which server should hold a specific piece of data (or key) is **Modulo Hashing**. Given a set of *N* servers, a hash function calculates a numeric value for a key, and we use the modulo operator to map it to a specific server index: `server_index = hash(key) % N`.

While this approach evenly distributes keys initially, it suffers from a critical flaw in dynamic environments where servers can scale up or down. If a server is added or removed, *N* changes. When *N* changes, the result of `hash(key) % N` changes for almost every single key in the system.

THE REHASHING CATASTROPHE

If you have 4 cache servers and one goes down (reducing *N* to 3), approximately 75% of your cached keys will suddenly map to a different server. This causes a massive wave of cache misses, overloading your backend database and potentially bringing down the entire system.

Impact of Topology Changes

Scenario	Servers (N)	Key 'User123' (Hash: 81)	Result
Initial State	4	81 % 4 = 1	Mapped to Server 1
Scale Up	5	81 % 5 = 1	Mapped to Server 1 (Lucky!)
Server Failure	3	81 % 3 = 0	Mapped to Server 0 (Cache Miss)

We need a routing strategy that minimizes data movement when the cluster size changes. This is where Consistent Hashing comes in.

2. Consistent Hashing Explained

Consistent Hashing solves the rehashing problem by independently mapping both the **servers** and the **keys** onto a conceptual circle, known as the *Hash Ring*.

The Hash Ring Concept

Imagine a hash function (like SHA-1 or MD5) that outputs a massive range of integers, for instance, from 0 to $2^{32} - 1$. Instead of imagining these numbers on a straight line, we wrap the line around so the largest value connects back to zero, forming a ring.

1. **Placing Servers:** We hash the identifier of each server (e.g., its IP address or a unique name). The resulting hash value dictates the server's position on the ring.
2. **Placing Keys:** When a request comes in for a specific key, we hash that key using the exact same hash function, which places it somewhere on the ring.
3. **Routing:** To find which server holds a key, we traverse the ring *clockwise* from the key's position until we encounter the first server. That server is responsible for the key.

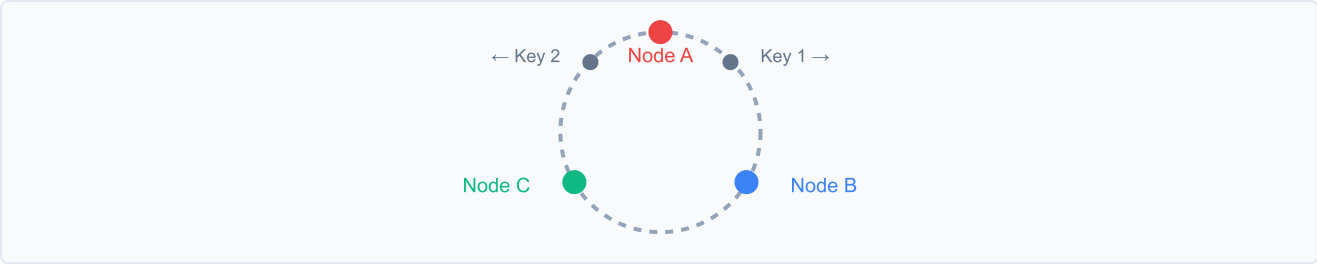


Figure 1: Keys are assigned to the next clockwise Node on the Hash Ring.

Graceful Scaling: If Node B goes down, only the keys that were assigned to Node B are remapped. Because we just walk clockwise, those keys naturally fall to the next server on the ring (Node C). The keys originally assigned to Node A and Node C stay exactly where they are. Data movement is strictly limited to *K/N* (total keys / total nodes).

3. Handling Imbalance: Virtual Nodes

A pure implementation of a hash ring often leads to a skewed distribution of keys. Hash functions generate pseudo-random outputs, meaning servers placed on the ring might clump together, leaving huge gaps. A server with a massive gap preceding it will end up owning a disproportionate amount of the keys, leading to hotspots.

To fix this, we introduce **Virtual Nodes (VNodes)**. Instead of placing each physical server onto the ring exactly once, we place it multiple times using slightly modified identifiers.

> **IMPLEMENTING VNODES**
>
> If we have Server A, we create replicas: `A#1`, `A#2`, `A#3`, ... `A#k`. We hash all *k* identifiers and place them on the ring. Now, Server A controls *k* smaller segments scattered evenly around the circle, statistically guaranteeing a uniform load distribution.

4. Implementation Data Structures

How do we efficiently "walk clockwise" in code? A naïve approach would require scanning every server, which is slow. We need a data structure that can quickly find the smallest value greater than or equal to the key's hash.

Approach 1: Binary Search Tree (BST)
We can store the server hashes in a balanced BST (like a Red-Black Tree). To find a server, we search the tree for the key's hash. If we find an exact match, great. Otherwise, we find the "ceiling" (the smallest element strictly greater than the key). Time complexity: O(log N).

Approach 2: Sorted Array + Binary Search (Most Common)
Since server topology rarely changes compared to the volume of incoming requests, we can prioritize read speed. We store all server hashes in a flat array and sort it. When a request arrives, we use a standard Binary Search (like `std::upper_bound` in C++ or `bisect` in Python) to find the server.

```
      def get_server(key, sorted_ring_array): if not sorted_ring_array: return None
key_hash = hash_function(key) # Binary search for the first server hash >= key_hash
index = bisect.bisect_left(sorted_ring_array, key_hash) # Wrap around if the key hash
is greater than all server hashes if index == len(sorted_ring_array): index = 0 return
ring_map[sorted_ring_array[index]]
```

Consistent hashing is a fundamental primitive used heavily in production systems like Apache Cassandra, Amazon DynamoDB, and modern CDNs to ensure stable, predictable scaling.